

Report

Setup 先記錄現在的時間，如果是 Tx 端的話就把現在的時間記錄進去，如果是 Rx 端的話，就把現在的時間減掉之前紀錄的 Tx 的時間來當作經過的時間

```
static void EchoTxRx (std::string context, const Ptr< const Packet > packet, const TcpHeader &header, const Ptr< const TcpSocketBase > socket)
{
    // TODO: Calculate end-to-end delay
    uint32_t seq = header.GetSequenceNumber().GetValue();
    Time now = Simulator::Now (); //現在的時間
    // Hint1: Parse the packet (you may refer context.find())
    // Hint2: Store send/arrival time for the same sequence number
    if(context.find("Tx") != std::string::npos){ //找context中有沒有Tx
        sendTimes[seq] = now; //如果是Tx發送端的話記錄現在的時間
    }
    // Hint3: Calculate end-to-end delay
    else if(context.find("Rx") != std::string::npos){
        if(sendTimes.find(seq) != sendTimes.end()){ //看sendTimes find會去找其中有沒有seq這個元素，有的話代表之前有存過時間，沒有會回傳sendTimes.end()
            Time delayTime = now - sendTimes[seq]; //存經過的時間
            delays[seq] = delayTime.GetSeconds(); //存到delays陣用秒來存
        }
    }
    std::cout << Simulator::Now () << ":" << context << ":" << packet->GetUid() << ":" << socket->GetNode () << ":" << seq << std::endl;
    //delay[1] = 2.5; // This value is for reference only
}
```

計算 average time 把全部的時間加總除以總數

```
// TODO: Output End-to-end Delay
double avg_delay = 0;
double sum_delay = 0;
for(auto &entry: delays){
    std::cout << "Packet " << entry.first << " end-to-end delay is " << entry.second << " s" << std::endl;
    sum_delay += entry.second; //加總全部傳的時間，上面會把每一個單獨seq傳的時間印出來
}
if (!delays.empty()){
    avg_delay = sum_delay / delays.size(); //算平均印出來
    std::cout << "Average end-to-end delay is " << avg_delay << " s" << std::endl;
}
```

可以把 8kbps 改成其他數字去更改 link data rate

```
utCh.SetGndDeviceAttribute("DataRate", StringValue("8kbps")); //改 link data rate的地方
```

Task1:

把經緯度改成現在要實作的，用 **GetObject**、**GetPosition** 先隨便找一個衛星的點來算半徑是多少，並利用球座標轉換公式得到該經緯度的衛星的 xyz 座標，gs 的話就直接用 **GetObject** 取出座標就好，最後輸出到檔案(node_positions)。

```
double gsLat = 20;
double gsLong = 0;
Vector pos0 = satellites.Get(0)->GetObject<MobilityModel>()->GetPosition(); //隨便取一個衛星來算半徑是多少
double r = sqrt(pos0.x * pos0.x + pos0.y * pos0.y + pos0.z * pos0.z); //半徑
double lat_rad = gsLat * M_PI / 180.0; //把經緯度轉換成弧度
double long_rad = gsLong * M_PI / 180.0;
double satX = r * cos(lat_rad) * cos(long_rad); //帶球座標轉換公式求得xyz座標
double satY = r * cos(lat_rad) * sin(long_rad);
double satZ = r * sin(lat_rad);
Vector gsPos = users.Get(0)->GetObject<MobilityModel>()->GetPosition(); //得到gs的座標
std::ofstream file("node_positions.txt");
if(file.is_open()){
    file << "GS " << gsPos.x << " " << gsPos.y << " " << gsPos.z << std::endl;
    file << "SAT " << satX << " " << satY << " " << satZ << std::endl;
}
file.close();
```

Task2:

先讀取座標檔案並存到 tx, rx 中，更改 tx_beam_direction = 0:5:90

```
filename = 'node_positions.txt';
fid = fopen(filename, 'r');
data = textscan(fid, '%s %f %f %f');
fclose(fid);
GS_coordinate = [data{2}(1), data{3}(1), data{4}(1)];
SAT_coordinate = [data{2}(2), data{3}(2), data{4}(2)];
tx_location = GS_coordinate;
rx_location = SAT_coordinate;
fprintf('Tx (GS) Coordinates: (%.2f, %.2f, %.2f)\n', tx_location);
fprintf('Rx (SAT) Coordinates: (%.2f, %.2f, %.2f)\n', rx_location);
```

用簡報中的公式去計算衛星所在的角度，最後取絕對值是因為後面有一筆資料角度算出來會是負的，因此取絕對值方便後面計算

```
dx = rx_location(1) - tx_location(1);
dy = rx_location(2) - tx_location(2);
d_h = sqrt(dx^2 + dy^2);
dz = rx_location(3) - tx_location(3);
theta_actual = abs(atan2(d_h / dz));
```

算 tx_gain 的部分跟作業一的算法一樣，就不再放上來，只有要把 cos 轉成 sin 因為現在 0 度代表的是上方，90 度代表的是水平。下面是計算 pathloss 的 code，這邊用 180 減掉 index 是因為前面是用 sin 去計算 tx_gain，且我把 resolution 改成 180，因 code book size 變成到 90，這樣才能保持 0.5 切一格，後面代公式計算 pathloss 輸出到檔案(pathloss_output)中。計算沒有 beamforming 的 code，只有把 tx_gain 的部分換成 1 去計算。

```
[~, idx_theta_actual] = min(abs(angles - theta_actual));
tx_gainuser1 = tx_gain1(180-idx_theta_actual);
rx_gain = 1;
dist1 = norm(rx_location - tx_location);
pluser1 = friis_equation(freq, tx_gainuser1, rx_gain, dist1);
```

Task3:

把 leo-propagation-loss-model.h 裡面的 DoCalcRxpower 換到 public 裡面

```
public:
    virtual double DoCalcRxPower (double txPowerDbm,
        Ptr<MobilityModel> a,
        Ptr<MobilityModel> b) const;
```

因為我的 pathloss 輸出檔中只有一個數字，因此用 getline 讀取一行轉成 double，並把 m_freeSpacePathLoss 更改成讀取進來的 pathloss，其中我把 bfpassloss 改成用加的，我看 lab1 跟網路上找之後，認為應該是加的才是正確的，因 pathloss 算出來基本上都是負的。(把兩個都改成實際值)

```
double rxc = txPowerDbm - 4.8 + bfpathloss - 0.36;
```

```
double bfpathloss = 0.0;
{
    std::ifstream inFile("pathloss_output.txt");
    if (!inFile.is_open())
    {
        NS_LOG_DEBUG ("Unable to open pathloss_output.txt. Using 0 dB as fallback.");
    }
    else
    {
        std::string line;
        if (std::getline(inFile, line))
        {
            pathlossFromFile = std::stod(line); //轉換成double
        }
        inFile.close();
    }
}
```

把 propagation-loss-model.h(src 裡)的 DoCalcRxpower 換到 public 裡面

```
public:
    virtual double DoCalcRxPower (double txPowerDbm,
                                   Ptr<MobilityModel> a,
                                   Ptr<MobilityModel> b) const = 0;
```

把 mock-channel.cc 裡面的-900 換成-120

```
rxPower = pLoss->CalcRxPower (txPower, srcMob, dstMob);
if (rxPower < -120.0)
```

Task4:

其中 gs 的座標是 gsX,gsY,gsZ，衛星座標是 sX,sY,sZ，都是拿前面算過的座標直接拿來用，之後建立 model 後 call 前面 task3 設置的 function 來算 rx_power，進而算出 SNR 改成 ratio 之後套 Shannon theorem 得到結果

$$R_{\max} = B \log_2 (1 + S / N),$$

```
//建立一個指標指向兩個固定不變的MobilityModel物件，用來當gs與衛星
Ptr<ConstantPositionMobilityModel> gsMob = CreateObject<ConstantPositionMobilityModel>();
gsMob->SetPosition(Vector(gsX, gsY, gsZ));
Ptr<ConstantPositionMobilityModel> satMob = CreateObject<ConstantPositionMobilityModel>();
satMob->SetPosition(Vector(sX, sY, sZ));
//建立一個LeoPropagationLossModel，並使用其中的函式DoCalcRxPower，傳入衛星跟gs的位置讓他計算rx_power
Ptr<LeoPropagationLossModel> propModel = CreateObject<LeoPropagationLossModel>();
double rxPowerDbm = propModel->DoCalcRxPower(txPowerDbm, satMob, gsMob);
double snrDb = rxPowerDbm - noiseDbm;
double snrLinear = std::pow(10.0, snrDb / 10.0);
// data rate的公式是 bandwidth * log2(1+S/N)
double dataRate = bandwidth * std::log2(1.0 + snrLinear);
cout << "RxPower    = " << rxPowerDbm << " dBm\n"
      << "SNR      = " << snrDb << " dB\n"
      << "DataRate  = " << dataRate << " bps\n\n";
```

Task5:

先讀取前面算出來的 data rate 當作 gs 發送到衛星的速率，然後找到我們傳輸的衛星，速率改成 1Gbps，我有更改 initial position 去找出我們實作的衛星 id 是多少，取經緯度相差很小的(我原本是找完全一樣的，但發現找不到，因此換成用差距)，而計算 end to end delay 在 setup 有做過了。

```
std::ostream oss;
oss << (uint64_t)dataRate << "bps";
utNet.Get(25)->GetObject<MockNetDevice>()->SetDataRate(DataRate(oss.str()));
utNet.Get(matchingSatelliteId)->GetObject<MockNetDevice>()->SetDataRate(DataRate("1Gbps"));

double tol = 1e-2;
if (fabs(lat - lat1) < tol && fabs(longit - longit1) < tol) {
    matchingSatelliteId = i;
    cout << "matching satellite id: " << i << endl;
}
```

Initial commit 我把 setup 的部分都實作完成

second commit 我把 task1、2 的部分完成

final commit 我把剩下的部分全部完成

最後一次 commit 是加上下面的更動

我的 data rate 的輸出是 bps，助教輸出應該是 kbps 所以差了 1000 倍。並且我直接把 m_atmosphericLoss 和 m_linkMargin 改成 4.8 跟 0.36，再把後面的步驟全部重新跑一次，我的 without beamforming 的平均時間是用 8kbps 跑的，沒有改 data rate